

Student Management System

Objective: Learn MongoDB **CRUD operations** using MongoDB **Compass and Shell** by creating a **Student Management System**.



Task 1: Setup MongoDB & Compass

1. **Open MongoDB Compass** and connect to your local MongoDB instance.
 2. **Create a new database** named `studentDB`.
 3. **Create a collection** named `students` inside `studentDB`.
-



Task 2: Insert Student Data

📌 Insert the following student records into the `students` collection.

Method 1: Using MongoDB Compass

- Open **MongoDB Compass** → Navigate to **studentDB** → Open **students** collection → Click **Insert Document**.

Copy and paste this JSON:

```
[
  {
    "name": "Alice Johnson",
    "age": 21,
    "email": "alice@example.com",
    "course": "Computer Science",
    "marks": [85, 90, 88],
    "city": "New York"
  },
  {
    "name": "Bob Williams",
    "age": 22,
    "email": "bob@example.com",
```

```
    "course": "Mechanical Engineering",
    "marks": [78, 83, 80],
    "city": "Los Angeles"
  },
  {
    "name": "Charlie Brown",
    "age": 20,
    "email": "charlie@example.com",
    "course": "Electrical Engineering",
    "marks": [92, 89, 95],
    "city": "Chicago"
  }
]
```

-
- Click **Insert** to save the documents.

Method 2: Using Mongo Shell

1. Open **Mongo Shell**.

Select `studentDB`:

```
use studentDB
```

- 2.

Insert data manually:

```
db.students.insertMany([
  {
    name: "Alice Johnson",
    age: 21,
    email: "alice@example.com",
    course: "Computer Science",
    marks: [85, 90, 88],
    city: "New York"
  },
  {
```

```
    name: "Bob Williams",
    age: 22,
    email: "bob@example.com",
    course: "Mechanical Engineering",
    marks: [78, 83, 80],
    city: "Los Angeles"
  },
  {
    name: "Charlie Brown",
    age: 20,
    email: "charlie@example.com",
    course: "Electrical Engineering",
    marks: [92, 89, 95],
    city: "Chicago"
  }
])
```

3.

Verify inserted data:

```
db.students.find().pretty()
```

4.

Task 3: Perform CRUD Operations

a) Read Data (Retrieve Students)

Retrieve all students:

```
db.students.find().pretty()
```

1.

Find a student by name (Alice Johnson):

```
db.students.find({ name: "Alice Johnson" }).pretty()
```

2.

Find students enrolled in Computer Science:

```
db.students.find({ course: "Computer Science" }).pretty()
```

3.

Retrieve students who are above 21 years old:

```
db.students.find({ age: { $gt: 21 } }).pretty()
```

4.

Find students who have scored more than 85 in any subject:

```
db.students.find({ marks: { $gt: 85 } }).pretty()
```

5.

Count the number of students in Mechanical Engineering:

```
db.students.countDocuments({ course: "Mechanical Engineering" })
```

6.

Sort students by age in descending order:

```
db.students.find().sort({ age: -1 }).pretty()
```

7.

b) Update Student Information

Update Bob's age to 23:

```
db.students.updateOne(
  { name: "Bob Williams" },
  { $set: { age: 23 } }
)
```

1.

Add a new field grade: "A" for Charlie:

```
db.students.updateOne(
  { name: "Charlie Brown" },
  { $set: { grade: "A" } }
)
```

2.

Increase Alice's marks by adding a new score (95):

```
db.students.updateOne(  
  { name: "Alice Johnson" },  
  { $push: { marks: 95 } }  
)
```

3.

Change Charlie's city to Boston:

```
db.students.updateOne(  
  { name: "Charlie Brown" },  
  { $set: { city: "Boston" } }  
)
```

4.

Increase all students' age by 1 year:

```
db.students.updateMany(  
  {},  
  { $inc: { age: 1 } }  
)
```

 **Verification:** Use `find()` queries to check if the updates worked.

c) Delete a Student Record

Remove Alice's record:

```
db.students.deleteOne({ name: "Alice Johnson" })
```

1.

Delete all students from Los Angeles:

```
db.students.deleteMany({ city: "Los Angeles" })
```

2.

Delete all students below 21 years of age:

```
db.students.deleteMany({ age: { $lt: 21 } })
```

3.

 **Verification:** Run `find()` to check if the records were deleted.